Behavior Tree (Selection Algorithms) Code Analysis

Repo: https://eae-git.eng.utah.edu/u1537288/playerstatepattern/-/tree/BehaviorTree?ref_type=tags

Analyze your usage of the AI behavior selection algorithm

I used a combination of a behavior tree for decision-making and the Type Object pattern for defining enemy behavior data. I chose this approach because it allowed me to structure enemy decision-making while keeping behaviors flexible and data-driven. Instead of using Unreal's built-in behavior trees, I implemented my own system using a custom 'AIController' and a 'BehaviorTreeNode' class hierarchy. This approach replaces Unreal's built-in AI tools and ensures all behavior selection logic is implemented manually in C++, as required by the assignment.

The AI evaluates conditions such as distance to the player, attack range, detection range, and cooldown state. Based on these checks, enemies select actions like patrolling, chasing, attacking, or retreating. The 'AIController' handles decision execution, while the 'BehaviorTreeNode' system organizes logic into selector and sequence nodes, making the behavior easier to structure and expand.

```cpp
class AEnemyBase;

enum class EBTNodeResult : uint8
{
    Success,
    Failure,
    Running
};

class BehaviorTreeNode
{
public:
    virtual ~BehaviorTreeNode() = default;
    virtual EBTNodeResult Tick(AEnemyBase* Enemy) = 0;
};

// ========================
// Composite Nodes
// ========================

class BTComposite_Selector : public BehaviorTreeNode
{
public:
    BTComposite_Selector() = default;
    BTComposite_Selector(const TArray<TSharedPtr<BehaviorTreeNode>>& InChild

    void AddChild(TSharedPtr<BehaviorTreeNode> Child);

    virtual EBTNodeResult Tick(AEnemyBase* Enemy) override;

private:
    TArray<TSharedPtr<BehaviorTreeNode>> Children;
};

class BTComposite_Sequence : public BehaviorTreeNode
{
public:
    BTComposite_Sequence() = default;
    BTComposite_Sequence(const TArray<TSharedPtr<BehaviorTreeNode>>& InChild

    void AddChild(TSharedPtr<BehaviorTreeNode> Child);

    virtual EBTNodeResult Tick(AEnemyBase* Enemy) override;

private:
```

```cpp
// ========================
// Condition Nodes
// ========================

class BTCondition_CanSeePlayer : public BehaviorTreeNode
{
public:
    virtual EBTNodeResult Tick(AEnemyBase* Enemy) override;
};

class BTCondition_PlayerInAttackRange : public BehaviorTreeNode
{
public:
    virtual EBTNodeResult Tick(AEnemyBase* Enemy) override;
};

// ========================
// Action Nodes
// ========================

class BTAction_Patrol : public BehaviorTreeNode
{
public:
    virtual EBTNodeResult Tick(AEnemyBase* Enemy) override;
};

class BTAction_ChasePlayer : public BehaviorTreeNode
{
public:
    virtual EBTNodeResult Tick(AEnemyBase* Enemy) override;
};

class BTAction_Attack : public BehaviorTreeNode
{
public:
    virtual EBTNodeResult Tick(AEnemyBase* Enemy) override;
};
```

In the future, I would make the node system more modular by separating conditions and actions into reusable components. Some logic is still tied too closely to the enemy class. However, I would not switch to Unreal's built-in behavior trees because implementing it manually helped me better understand AI systems and met the assignment requirements.

```cpp
void AEnemyBase::BuildBehaviorTree()
{
    if (!EnemyType)
    {
        UE_LOG(LogTemp, Warning, TEXT("%s: Missing EnemyType"), *GetName());
        return;
    }

    TSharedPtr<BTComposite_Selector> RootSelector = MakeShared<BTComposite_Selector>();

    if (EnemyType->bCanAttack)
    {
        TSharedPtr<BTCondition_PlayerInAttackRange> InAttackRangeNode = MakeShared<BTCondition_PlayerInAttackRange>();
        TSharedPtr<BTAction_Attack> AttackNode = MakeShared<BTAction_Attack>();

        TSharedPtr<BTComposite_Sequence> AttackSequence = MakeShared<BTComposite_Sequence>();
        AttackSequence->AddChild(InAttackRangeNode);
        AttackSequence->AddChild(AttackNode);

        RootSelector->AddChild(AttackSequence);
    }

    if (EnemyType->bCanChase)
    {
        TSharedPtr<BTCondition_CanSeePlayer> CanSeePlayerNode = MakeShared<BTCondition_CanSeePlayer>();
        TSharedPtr<BTAction_ChasePlayer> ChaseNode = MakeShared<BTAction_ChasePlayer>();

        TSharedPtr<BTComposite_Sequence> ChaseSequence = MakeShared<BTComposite_Sequence>();
        ChaseSequence->AddChild(CanSeePlayerNode);
        ChaseSequence->AddChild(ChaseNode);

        RootSelector->AddChild(ChaseSequence);
    }

    if (EnemyType->bCanPatrol)
    {
        TSharedPtr<BTAction_Patrol> PatrolNode = MakeShared<BTAction_Patrol>();
        RootSelector->AddChild(PatrolNode);
    }

    RootBehaviorTree = RootSelector;
}
```

Analyze your usage of other patterns (not the AI behavior selection alg)

The patterns I used the most were the State Pattern for the player and the Type Object pattern for enemies.

I was able to build off my State Pattern very easily, with only having to identify the state, then implement it on the player character script. Unreal allows me to easily create an input action mapping and assign it to the player character in the editor. The hardest part before using this

pattern was getting the animation graph to work properly, but thanks to enums, I can easily check states with the appropriate connections.

The Type Object pattern was implemented through 'UEnemyType' data assets and a shared 'AEnemyBase' class. Enemy differences such as stats, attack type, and assets are defined in data, while the base class executes the behavior. The behavior system, player state system, and enemy data system all work together, which shows how multiple patterns can seamlessly and even benefit from working together.

```cpp
/* ========================
   Attack
   ======================== */

void AttackState::Enter(AMyCharacter* Character)
{
    Character->SetMovementState(EMovementState::Attack);
    Character->SetInvulnerable(false);
    Character->GetCharacterMovement()->MaxWalkSpeed *= 0.1f
    Character->PlayAttackMontage();
    Character->PlayAudio();
}

void AttackState::Exit(AMyCharacter* Character)
{
    Character->SetInvulnerable(false);
    Character->ApplyStanding();
}
```

```cpp
/* ========================
   Dodge Roll
   ======================== */

void DodgeRollState::Enter(AMyCharacter* Character)
{
    Character->SetMovementState(EMovementState::DodgeRoll);

    UCharacterMovementComponent* MoveComp = Character->GetCharacterMovement();
    if (MoveComp)
    {
        Character->SavedGroundFriction = MoveComp->GroundFriction;
        Character->SavedBrakingDecelerationWalking = MoveComp->BrakingDecelerationWalking;

        MoveComp->GroundFriction = 1.f;
        MoveComp->BrakingDecelerationWalking = 150.f;
    }

    Character->PlayAudio();
}

void DodgeRollState::Exit(AMyCharacter* Character)
{
    Character->SetInvulnerable(false);
    Character->bDodgeLocked = false;


    UCharacterMovementComponent* MoveComp = Character->GetCharacterMovement();
    if (MoveComp)
    {
        MoveComp->GroundFriction = Character->SavedGroundFriction;
        MoveComp->BrakingDecelerationWalking = Character->SavedBrakingDecelerationWalking;
    }
}
```

I would refine the State Pattern by reducing repeated transition logic and possibly adding shared functionality between states. I would also further separate attack behaviors in the Type Object system. However, I would keep both patterns because they made the systems more organized, scalable, and easier to maintain.

What pattern was the most critical to your design?

The Type Object pattern was the most critical because it worked directly with the behavior selection system. By defining enemy stats and behavior data in 'UEnemyType', the custom 'AIController' and 'BehaviorTreeNode' system could use that data to determine how each enemy should act at runtime. This allowed a single enemy class to support multiple behaviors without

creating separate subclasses. It made the AI system more flexible and scalable, since changing an enemy's behavior could be done through data rather than modifying code.

How could you do it differently?

I could have used separate subclasses for each enemy type instead of a data-driven approach. However, that would have made the system harder to scale and maintain, especially as more enemy types were added. The Type Object pattern kept the design cleaner and better integrated with the behavior system. (I plan on revisiting the subclass sandbox for different weapons)

What pattern(s) did you choose not to use? Why didn't you use them? Justify your response

I chose not to use a State Machine or a Decision Tree for enemy AI. A basic state machine would have limited flexibility, as it can become difficult to manage when many transitions and conditions are involved. A decision tree, while simpler, would not scale well for more complex behaviors or allow for easy reuse of logic. Opting for a Behavior Tree allowed for more complexity, which provided a more structured and modular way to handle decision-making. This approach allowed me to reuse logic through selector and sequence nodes and made it easier to expand enemy behaviors.

What valuable information did you learn for yourself while doing this project? What did you find difficult? What was easy?

I learned that implementing a behavior tree alone is somewhat limiting, but integrating it with a type object makes it not only designer-friendly, but incredibly modular. From only a handful of actions, you can mix and match to make several different enemies pretty fast (although some combinations won't make sense). I also learned a lot of other stuff not related to AI behaviors or design patterns, like things related to animations, shaders, and environment design (In general, a lot of Unreal's built-in tools, i.e., landscaping, Foliage, modeling). The behavior tree was the easy part. Figuring out how to design a Dark Souls-like level so the player doesn't get stuck or break out of the map took a lot of playtesting/trail and error.

Time Spent

I spent well over 35 hours, as I wanted to get a good foundation for my game in preparation for a portfolio piece as well as the final project. I have structured my project in ways where I can add other patterns or systems, like using GAS for combat, and also creating a boss battle in a Dark Souls Style. I also want to add an upgrade stats system when resting at the bonfire.